AmberMonkey:

DRAFT
draft@draft.ca
University of Alberta
Edmonton, Canada

DRAFT
draft@draft.ca
University of Alberta
Edmonton, Canada

DRAFT
draft@draft.ca
University of Alberta
Edmonton, Canada

JavaScript engines provide restricted-execution modes to disable guest-controlled just-in-time (JIT) compilation. Restricted execution aims to reduce an engine's attack surface, howeever, disabling JIT compilation can substantially reduce performance. We find that inline-cache (IC) bodies, among other Baseline JIT artifacts, can be compiled ahead of time (AOT) to reclaim JIT performance lost under restricted settings. Our analysis shows that dynamic IC requests concentrate in a small recurring set of stub bodies, making their inclusion into a fixed AOT image feasible. Our 732 KB corpus serves 99.6% of IC-body requests on Speedometer 3.1 and 98.3% on JetStream 3.0.

We present AmberMonkey, a code-generation model that reuses SpiderMonkey's Baseline JIT to produce ahead-of-time native code. AmberMonkey uses a two stage build to bootstrap AOT relocated artifacts into a native image which can be compiled into the final FireFox release. JIT-generated instructions typically embed pointers to data and code of the enclosing JavaScript runtime, coupling an artifact to a single process. AmberMonkey characterizs the nature of JIT coupling and provides relocation schemes. We defer a portion of symbol relocations to the native program linker, while a *Runtime Indirection Table* (RIT) makes runtime symbols attainable through indirection at a 7% overhead. Under restricted exection AmberMonkey improves throughput by 51% over interpretation on Speedometer 3.1 and reachs 57% of default tiered-JIT throughput. Finally, our immutable AOT image avoids redundant JIT compilation across runtimes, reducing per-content-process engine PSS by 57% on Speedometer 3.1.

I. INTRODUCTION

Restricted-execution modes prevent untrusted guest JavaScript from invoking just-in-time (JIT) compilation. Platforms adopt these modes to reduce the attack surface exposed by compiling guest-controlled JavaScript. A 2026 review of V8 bugs estimated that JITs accounted for roughly 50% of the tracked vulnerabilities [1]. Disabling the JIT, however, can significantly reduce application throughput. V8 initially reported a 40% Speedometer 2.0 slowdown in its JITless configuration [2]. Similarly, SpiderMonkey relies solely on its generic interpreter, which achieves only 38% of optimized performance on Speedometer 3. The gap between interpreted and JIT-optimized performance motivates an ahead-of-time (AOT) code generation model to recover throughput without guest-directed compilation.

JavaScript's dynamic typing has restricted efforts to provide type-specialized native code in an AOT setting. Prior systems have therefore emphasized compilation without runtime type profiles. Hopc, for example, compiles complete JavaScript programs with a standalone AOT compiler [3]. The first Futamura projection offers another route: specializing an interpreter for a fixed program derives a compiled implementation of that program [4]. Deriving compilation from an interpreter also keeps the interpreter as the source of truth for language semantics rather than requiring a separate compiler backend. Weval applies this projection to SpiderMonkey's Portable Baseline Interpreter (PBL) and JavaScript bytecode stored in a WebAssembly snapshot [5]. It specializes PBL's interpreter loops for both JavaScript bytecode and CacheIR, producing WebAssembly functions for guest functions and corpus IC bodies known during snapshot processing. The result is analogous to Baseline compilation: it eliminates interpreter dispatch while leaving run-time type specialization to inline caches (ICs).

While these approaches differ in implementation, both derive their core performance improvmenets by assuming the guest program is available during AOT compilation. In contrast, a browser engines workload is dynamic. Our AOT design is therefore constrained to assume that JavaScript arrives after our executable image has been fixed. Restricted execution prohibits mapping any guest controlled data into executable memory at runtime.

Even for typical environments where all JavaScript could be Baseline compiled, eagerly materializing all functions is still avoided. Across 15,000 popular web pages, half of the JavaScript files studied contained at least 70% unused functions, accounting for 55% of their size [6]. Production engines consequently defer work for functions that may never execute. Lazy execution is so effective for modern JavaScript that SpiderMonkey avoids generating bytecode for many functions until their first execution, while V8 similarly defers full parsing and bytecode compilation [7], [8].

These workload constraints raise two questions for AOT compilation in browser engines. First, which native artifacts are known before deployment or recur often enough across unrelated programs to justify inclusion in a bounded image? Second, can artifacts produced by an existing JIT generator execute in independently initialized runtimes despite embedding process-specific code and data addresses? A practical design must both select artifacts without knowing the deployed workload and remove their dependence on the runtime that generated them.

To answer the first question, we characterize reuse among Baseline-tier artifacts. The Baseline Interpreter is deterministic for a fixed engine build, and SpiderMonkey's self-hosted JavaScript is available at build time. Application Baseline functions instead encode guest bytecode and rarely share exact identities across unrelated workloads. IC bodies recur because CacheIR separates executable bodies from the per-site data that specializes them [9]. This separation already enables recurring bodies to be shared within one process; a fixed corpus can extend that reuse across processes and workloads. We construct the corpus from the first three-quarters of

Mozilla's tp6 page set, which contains 32 sites, and retain the final quarter for held-out evaluation [10].

Per the second question, we are highly motivated to reuse existing code generation routines as to avoid introducing additional interpreations of the JavaScript specification. We elaborate on the security implications of doing so in section II.A. AmberMonkey runs SpiderMonkey's production Baseline generators during a trusted build, preserving their implementation of JavaScript semantics and their integration with existing engine interfaces. It precompiles the Baseline Interpreter, self-hosted Baseline functions, and a corpus of IC bodies. SpiderMonkey still observes types and selects IC specializations at run time, but matching native bodies come from an immutable image rather than run-time code generation.

To answer the second question, we make generated artifacts independent of the process that produced them. JIT-generated instructions normally embed addresses of data structures and code entry points created for one runtime. These addresses prevent the same native bytes from executing in a separately initialized runtime. We call an artifact *runtime-independent* when it outlines these dependencies so that separately initialized runtimes can use identical image bytes.

AmberMonkey classifies each embedded address by when it can be resolved. It rewrites some references as program-counter-relative control flow and delegates references to private ELF symbols to the native linker. For addresses available only after runtime initialization, a per-runtime *Runtime Indirection Table* (RIT) provides stable indirections. A recording build captures code from the existing generators and converts recognized runtime pointers into these relocatable forms. AmberMonkey then serializes the instructions and metadata into a native image that the loader exposes through SpiderMonkey's existing JIT interfaces. This design turns transient JIT output into persistent build artifacts without requiring high-level code generators to support a separate AOT backend.

On workloads excluded from corpus construction, the fixed corpus serves 99.6% of IC-body requests on Speedometer 3.1 and 98.3% on JetStream 3.0. Our AOT hardened configuration combines this corpus with the AOT Baseline Interpreter and 236 self-hosted Baseline functions. Together, these artifacts improve Speedometer 3.1 throughput by 51% over bytecode-only execution and reach 57% of default tiered-JIT throughput. We also measure the run-time cost of indirection and the binary-size cost of embedding the corpus.

The immutable AOT image also changes how memory scales across runtimes. Run-time-generated Baseline and IC code occupies private anonymous mappings. AmberMonkey instead embeds selected artifacts as file-backed text, allowing runtimes to reuse one corpus and processes to share its physical pages. V8′s embedded builtins similarly use file-backed instructions to eliminate per-isolate builtin copies [11]. This benefit became more important as site isolation increased renderer-process counts after Spectre [12]. On Speedometer 3.1, AmberMonkey reduces per-content-process engine PSS by 57% relative to runtime-generated Baseline code. This paper makes the following contributions:

1. We characterize cross-workload reuse among SpiderMonkey Baseline-tier artifacts. Across the 8 characterized tp6 sites, IC-body identities collected from one site cover a median 98% of another site's dynamic IC-body entries. Baseline-function identities cover only 2%. This contrast supports using operation-level IC bodies as the unit of a bounded cross-workload corpus.

2. We design and implement AmberMonkey, which turns an existing Baseline JIT into an AOT producer without adding another JavaScript compiler or interpreter. AmberMonkey classifies embedded address targets by resolution time, binds them through native linking or per-runtime indirection, and packages instructions and metadata in a native image. Its loader reconstructs SpiderMonkey's existing JIT objects around image-backed instruction ranges.

3. We evaluate AmberMonkey in SpiderMonkey on x86-64. Its fixed corpus serves 99.6% of IC-body requests on Speedometer 3.1 and 98.3% on JetStream 3.0. In the AOT-only configuration, AmberMonkey improves Speedometer 3.1 throughput by 51% over bytecode-only execution and reaches 57% of default tiered-JIT throughput. We also quantify indirection overhead, binary-size cost, and private JIT memory.

II. THE COMPLEXITY OF OPTIMIZING JAVASCRIPT

Before venturing to design a code-generation model to improve the performance of SpiderMonkey under restricted execution, we must first understand why JIT compilation is involved in security vulnerability. We identify the intricate semantics of the JavaScript specification, complexified by aditional optimziations, as the primary source. Notably this complexity applies to both interpreters and JIT compilers, motivating our design to reuse existing code-generators and limit the tier to Baseline.

A. Interpretation and Compilation

JavaScript just-in-time (JIT) compilers expose a direct path from attacker-controlled inputs to native instructions. An attacker can shape both the bytecode being compiled and the run-time profiles that drive speculative optimization. A missed side effect or invalid type assumption can cause the compiler to emit instructions that violate the engine's memory-safety invariants. Notably, implementing the compiler in a memory-safe language does not prevent these logic errors, as generated instructions perform the invalid access.

Recently automated bug finding has exposed security defects at scale. Mozilla reported that an early evaluation of Anthropic's Mythos Preview identified 271 vulnerabilities fixed in Firefox 150 [13]. One month later, Nebula Security reported CVE-2026-10702 [14]. The high-severity vulnerability arose because SpiderMonkey's optimizing compiler modeled an instruction that could allocate as a load. Global value numbering could then reuse a stale pointer to object-property storage. The resulting JIT miscompilation enabled arbitrary code execution in the content process [14].

This example shows that optimized just-in-time (JIT) compilation remains a source of vulnerabilities. However, treating

JIT compilation as the sole source of semantic bugs is a misconception. JavaScript's intricate semantics challenge both JIT compilers and interpreters; JIT compilers compound this complexity with optimizations that must preserve semantics. Microsoft's DrumBrake WebAssembly interpreter, developed for JIT-disabled execution, illustrates how interpreters can also fail. Researchers reported 23 remote-code- execution vulnerabilities involving type confusion, reference-stack indexing, missing garbage-collection barriers, and incorrect control-flow handling [15]. Although DrumBrake executes WebAssembly rather than JavaScript, these failures show that replacing compilation with interpretation can shift, rather than eliminate, the attack surface.

From these examples, AmberMonkey derives two design constraints. First, we reuse existing code generators rather than introduce another interpretation or compilation layer. Second, we reuse only Baseline JIT artifacts from trusted sources, excluding optimized artifacts and artifacts from untrusted sources. Discussion in [1] identified Baseline JIT compilation to be less frequently involved in exploits, primarily due to it's simple code-generation model. These constraints limit the possibility that our ahead-of-time (AOT) artifacts contain latent bugs.

B. Restricted Execution

Restricted-execution policies remove guest-controlled compilation from the run-time execution path. These policies differ in which other forms of code generation they permit.

In this paper, restricted execution means that guest JavaScript cannot invoke Baseline-function, inline-cache (IC), or optimizing compilation. Our evaluated configuration disables regular-expression and WebAssembly compilation but retains deterministic trampolines and entry preambles generated during trusted initialization.

AmberMonkey performs selected compilation in a build-controlled phase. Its ahead-of-time (AOT) image is fixed before guest execution, so a guest can neither add artifacts nor modify their instructions. Capture can preserve generator defects, and guests can still exercise defects in captured artifacts or other run-time components. AmberMonkey limits capture to Baseline-tier artifacts and excludes optimizing code. A 2026 review of known exploitable V8 bugs estimated that JITs accounted for roughly 50% of the tracked vulnerabilities but observed that Baseline JITs were rarely their source [1]. These V8-specific data motivate this boundary, but they do not establish that Baseline tiers in V8 or other engines are free of exploitable defects.

III. STRUCTURED IC BODIES AND REUSABLE AOT ARTIFACTS

SpiderMonkey's Baseline tier combines type-generic execution with operation-level specialization through CacheIR. This section describes the existing execution pipeline, identifies CacheIR as an ahead-of-time (AOT) reuse boundary, shows how AOT bodies retain run-time specialization, and measures whether their identities recur across unrelated workloads. Figure 1 previews the separation between a shared native body and the private fields that specialize each IC site.

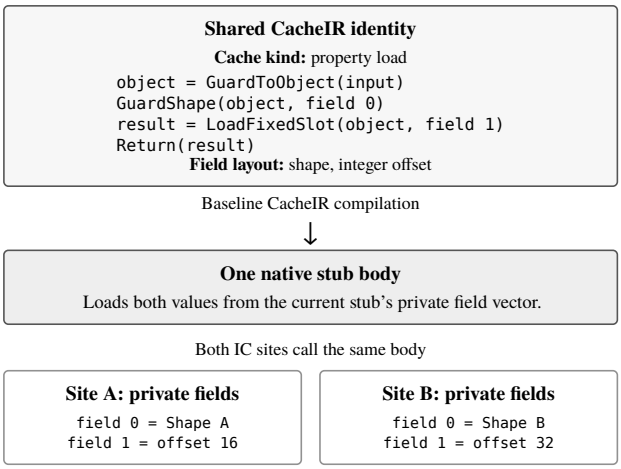

Figure 1: CacheIR separates a fast path's structural identity from its site-specific values. The Baseline CacheIR compiler emits one native body for the shared program and field layout. Each IC site supplies a private field vector when it executes that body. Program syntax and values are schematic.

A. SpiderMonkey's Baseline Tier

SpiderMonkey executes JavaScript through four tiers. An interpreter first executes a script's bytecode without generating guest-specific machine code. As the script becomes hot, SpiderMonkey advances through the generated Baseline Interpreter, per-script Baseline compilation, and optimizing Ion compilation [16]. AmberMonkey targets the two Baseline tiers and the IC bodies that provide their run-time specialization.

Baseline Interpreter: Each SpiderMonkey runtime generates one native Baseline Interpreter during initialization. It contains a type-generic native handler for each JavaScript bytecode opcode and dispatches between handlers while executing a script. At each cacheable opcode, the handler enters that site's IC chain. Every script in the runtime can therefore share one generated interpreter while retaining its own IC chains.

Inline-cache bodies: IC chains provide run-time specialization for both the Baseline Interpreter and Baseline-compiled functions. A fallback stub handles a chain miss and may use a CacheIR generator to describe guards and a fast path for the observed case. The Baseline CacheIR compiler translates this description into an executable stub body. SpiderMonkey stores site-specific fields, including shapes and slot indices, separately from that body, allowing multiple IC sites to reuse one compiled implementation [9], [17]. AmberMonkey includes the executable body in its AOT image while each runtime retains its own fields and chain metadata.

Baseline compilation: After an individual script becomes sufficiently hot, the Baseline compiler translates its complete bytecode sequence into per-script native code. It shares per-opcode generation infrastructure with the Baseline Interpreter, eliminates bytecode-dispatch overhead, and performs limited local stack tracking. The compiler consumes bytecode rather than run-time type profiles and relies on IC chains for

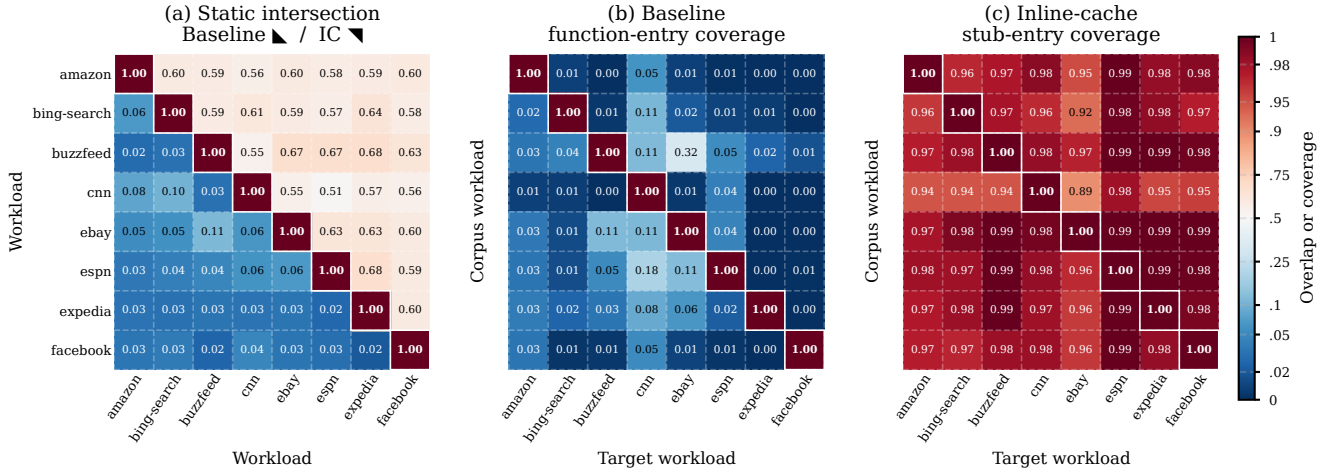

Figure 2: Cross-workload reuse across the first 8 tp6-Train workloads. Panel (a) reports Jaccard overlap between identity sets, with Baseline functions below the diagonal and inline-cache (IC) bodies above. Panels (b) and (c) report the fraction of target dynamic entries covered by identities from the row workload.

dynamic specialization [16]. We call the resulting program-specific artifact a *Baseline function*. Its instruction identity depends on immutable script data and the compilation configuration, but not on a run-time type profile.

B. CacheIR as an AOT Reuse Boundary

Structured inline caches expose operation-level native bodies as potential units of AOT reuse. SpiderMonkey's CacheIR gives these bodies a structural identity while retaining site-specific values in private data.

CacheIR makes Baseline IC bodies amenable to AOT sharing by separating the structural inputs to code generation from per-site specialization data. For a fixed engine build and code-generation configuration, the cache kind and CacheIR program determine the native body. The concrete field values and the IC site's script location do not. AmberMonkey serializes the field types with the body to reconstruct its data layout in a later runtime. Exact structural matching therefore provides a reuse key without manually maintained stub-code sharing keys. It does not merge semantically equivalent but structurally distinct CacheIR programs.

CacheIR represents this separation through three operand classes. Operand identifiers denote run-time inputs and intermediate values. Stub fields hold values fixed for one stub instance, such as an object shape or slot index. Immediates form part of the CacheIR program and therefore contribute to its structural identity [9], [17].

Figure 1 shows two property-load stubs with the same CacheIR program. The shape and slot offset differ, but both stubs place those values at the same field offsets. The Baseline CacheIR compiler emits instructions that load the values from the current stub. Both sites can therefore call the same native body. In contrast, SpiderMonkey's optimizing IC compiler can embed field values in instructions to avoid these loads, which prevents body sharing [9].

C. AOT Specialization

Run-time observations still select a CacheIR program and its private field values. AmberMonkey uses the program's structural identity to query the fixed AOT corpus. A hit combines the image-backed body with run-time-private fields; a miss continues on the generic fallback path without generating instructions. Untrusted execution can therefore select and parameterize precompiled bodies, but cannot extend the executable corpus.

CacheIR makes body reuse possible, but it does not guarantee that a bounded corpus will cover future execution. This utility depends on whether dynamic execution concentrates in a recurring set of structural identities across unrelated workloads.

D. Cross-Workload Reuse

We compare exact identities of operation-level IC bodies and complete Baseline functions to determine which granularity supports a bounded cross-workload corpus.

We partition the 32 desktop workloads in Mozilla's tp6 page-load suite alphabetically [10]. The first 24 form *tp6-Train*; the remaining 8 form the held-out *tp6-Test*. The pairwise study in Figure 2 uses the first 8 tp6-Train workloads. This deterministic subset was fixed without reference to overlap. We collect repeated cold page loads and retain content-process events.

We instrument SpiderMonkey to record Baseline-function identities, attached IC-body identities, and entries into both artifact types. Static overlap is the Jaccard index of two identity sets. Dynamic coverage is the fraction of a target's function or stub entries whose identity occurs in the corpus workload.

Baseline functions have a median Jaccard overlap of 0.04, and a workload covers only 2% of another's function entries at the median. For IC bodies, these values rise to 0.6 and 98%. The shared IC set therefore contains the frequently entered bodies: 51 of 56 pairs cover at least 95% of target stub entries, and the minimum is 90%.

Exact reuse decreases as artifacts incorporate more application context. These results support IC bodies, rather than application Baseline functions, as the workload-derived corpus unit. Section VI-B describes construction of the evaluated image; we do not measure trace or optimizing code.

Dependency	Examples	Resolution
Artifact-local	Basic blocks, handlers, return sites	Relative artifact offset
Engine symbol	Helpers, classes, C++ ABI functions	Native linker
Runtime address	Contexts, caches, embedder symbols, permanent atoms	Per-runtime table
Deterministic runtime code	VM wrappers, barrier stubs, debug and exception tails	Trusted initialization
Unsupported	Movable GC cells and retargetable pointers	Reject artifact

Table 1: Dependency resolution for Baseline artifacts. Runtime-generated targets are deterministic engine infrastructure created during trusted initialization, not guest-compiled code.

IV. AMBERMONKEY DESIGN

AmberMonkey makes runtime pointers that JIT code ordinarily embeds directly available through a per-runtime side table, the *Runtime Indirection Table* (RIT). AOT artifacts access RIT entries at deterministic offsets that remain stable across processes. Each runtime populates its RIT during JIT initialization, allowing one AOT artifact to execute in multiple runtimes. AmberMonkey resolves other external addresses through artifact-relative references or native-linker relocations. This section describes these resolution methods, how shared code accesses the current runtime's RIT, and which runtime values the table can safely contain. We first illustrate the runtime coupling that requires these mechanisms, then explain how dynamic instrumentation affects code sharing.

A. Runtime Coupling

Firefox executes untrusted web JavaScript in sandboxed content processes [18]. Each process maps the same engine library, but SpiderMonkey associates generated Baseline infrastructure and IC state with a particular JavaScript runtime. AmberMonkey must preserve this private state while moving selected executable bodies into the shared library.

Baseline generation embeds concrete addresses of runtime-specific state in native instructions. Panel (a) of Listing 1 shows a simplified Baseline generator that obtains the current runtime's interrupt flag and passes its address to `ImmPtr`.

During generation, `ImmPtr` converts the generating runtime's flag address into an immediate machine-code operand. Copying the emitted instructions into a separately initialized process preserves that numeric address, which does not identify the destination runtime's flag. The generated check requires the current runtime's flag, not its address in the generating process. We call this mismatch between a stable semantic role and its runtime-specific representation *runtime coupling*.

B. Resolving External Addresses

We resolve each external address at the earliest stage that can identify it. In the simplest case, artifact-local branches use PC-relative displacements that remain valid when the artifact moves. AmberMonkey emits native-linker relocations for engine functions and data with hidden visibility. Other shared objects cannot interpose these definitions, so Firefox's linker can resolve the references within the final library. Values that depend on a particular runtime instead occupy fixed slots in its RIT. JIT initialization populates these slots after creating the runtime-specific state and generated code. Table 1 summarizes these resolution methods.

The RIT resembles an ELF global offset table (GOT) because both store absolute addresses in private data while keeping position-independent text shareable [19]. A conventional GOT names symbols that a linker or loader can resolve. Its standard relocations cannot name per-runtime contexts, runtime caches, or JIT-generated entry points because these values have no linkable symbols. We assign each such role a stable slot, then give every runtime a RIT with the same layout and its own addresses.

C. Accessing the RIT

A shared artifact must locate the RIT without embedding a direct reference to the table in its image. We considered a dedicated register, thread-local storage (TLS), and SpiderMonkey's existing frame layouts. A dedicated register would compete with registers already reserved for interpreter and IC state. TLS could recover the current runtime, but a fixed TLS displacement would couple the offline image to the platform's TLS layout and dynamic-library loading order. AmberMonkey instead stores the RIT base at a fixed offset in the active frame, while each table entry remains at a deterministic slot offset.

A Baseline frame records execution state for one invocation of the Baseline Interpreter or a Baseline-compiled function. Its fields remain at fixed offsets from the frame pointer regardless of which artifact is executing. AmberMonkey adds the RIT base to this layout, allowing shared code to recover it with one load while the Baseline frame remains active.

Inline-cache (IC) stubs initially execute with their caller's Baseline frame active and therefore use the same RIT-base field. Some stubs call runtime helpers that can trigger garbage collection. When a helper requires such a call, SpiderMonkey creates a stub frame so stack walking can map the return address to its bytecode location. Although a saved pointer keeps the caller's Baseline frame reachable, obtaining the RIT base through it would require two dependent loads. AmberMonkey copies the base into the stub frame during the transition, allowing stub code to recover it from the active frame with one load.

Every path that creates or reconstructs a Baseline frame initializes its RIT-base field before shared code executes. At ordinary AOT entry, a runtime-specific preamble loads the base into a temporary register and jumps to the image-backed artifact. The artifact prologue then stores the base in its Baseline frame. Runtime-generated Baseline code initializes the same field because it may enter an image-backed IC stub. Likewise, bailout, on-stack replacement, and generator-resume paths initialize the field when reconstructing a frame.

Using a frame field avoids dedicating a register but repeats the base load for each RIT access. V8 instead reserves a root register for isolate-relative accesses [11]. SpiderMonkey faces the same register-pressure trade-off for the Baseline Interpreter's bytecode program counter. While most architectures keep the program counter in a dedicated register, the 32-bit x86 backend stores it in the frame because it lacks spare registers. Our x86-64 implementation retains SpiderMonkey's existing register contracts across Baseline code and IC stubs. Because AArch64 provides a larger general-purpose register set, a future port could evaluate pinning

(a) Baseline code generator

```
void emitCheck(Masm& masm, Reg out)
{
  auto* flag = cx->interruptFlag();
  masm.movePtr(ImmPtr(flag), out);
  masm.branch32(NonZero,
    Address(out, 0), &interrupt);
}
```

(b) Simplified lowering

```
void movePtr(ImmPtr p, Reg dst) {
  if (auto slot = findSlot(p)) {
    emitLoadSlot(*slot, dst);
    return;
  }
  reportUnknownPointer(p);
}
```

(c) Schematic x86-64

```
# Before
movabsq $flag, %rax
cmpl    $0, (%rax)
# After
movq -24(%rbp), %r11  # table
movq  48(%r11), %rax # slot
cmpl  $0, (%rax)
```

Listing 1: Simplified illustration of transparent runtime-pointer lowering. Panel (a) represents Baseline Interpreter code generation with no AOT-specific logic. The MacroAssembler reverse-maps its direct pointer in panel (b) and emits the RIT access shown in panel (c). Identifiers and offsets are schematic.

the RIT base. We have not measured whether eliminating the base load would offset the resulting register pressure.

The frame-based design adds one machine word to each Baseline frame and materialized stub frame, as well as one base load to each generic RIT access. A value lookup then loads the selected slot, while reading mutable storage through that address requires a third dependent load. On x86-64, however, an indirect call or jump can fold the slot read into the control-transfer instruction.

D. Capturing Runtime Pointers

AmberMonkey classifies embedded pointers at the MacroAssembler boundary, keeping AOT capture transparent to higher-level code generators. During capture, intercepted pointer operations reverse-map each concrete address against the finite whitelist used to populate the RIT. A match selects either a RIT slot or a native-linker relocation. An unrecognized pointer causes an AOT validation build to report the missing entry and stop. Baseline and IC generators can therefore continue to issue ordinary MacroAssembler operations without AOT-specific logic. Listing 1 illustrates this interception and the resulting RIT access.

An address qualifies for the whitelist only if it remains fixed while its runtime can execute AOT code. The address may differ across runtimes, and the storage it names may remain mutable. For example, a RIT slot can hold the stable address of a nursery cursor or profiler flag whose contents change during execution. We call such an address *runtime-stable*.

Garbage-collected pointers additionally require a lifetime guarantee because the immutable artifact cannot trace or update them. A SpiderMonkey atom is an interned `JSString` cell allocated in a dedicated atoms zone. AmberMonkey admits seven permanent common-name atoms, including the interned strings `"true"`, `"false"`, `"null"`, and `"undefined"`. SpiderMonkey creates these atoms during initialization and neither moves nor collects them. AmberMonkey excludes other garbage-collected pointers that may move or be reclaimed.

E. Dynamic Instrumentation

SpiderMonkey uses patchable code to enable JavaScript debugging at run time. The generated Baseline Interpreter contains debugger paths even in release engine builds. This instrumentation supports debugging guest JavaScript and is independent of native debugging support for the engine binary. A `toggledJump` initially skips each path. On x86-64, enabling instrumentation changes the jump into a same-sized comparison that falls through to the instrumented path. Breakpoint and stepping sites separately replace patchable NOPs with calls to the debug-trap handler.

AmberMonkey retains these patch sites in the image-backed Baseline Interpreter. It does not install an image-backed Baseline function for a script already marked as a debuggee. SpiderMonkey instead generates private code containing the required debug instrumentation. Profiling follows a similar split. Registering an image-backed range in the profiler's native-to-bytecode map does not modify its instructions, whereas enabling profiler entry and exit instrumentation patches its toggled jumps.

Patching an image-backed range reduces its sharing benefit. AmberMonkey temporarily changes the containing `.text.aot` pages from read-execute to read-write and restores read-execute permissions afterward. The first write gives the process a private copy of each affected page. Debugging or profiling therefore forfeits cross-process sharing for those pages, while unmodified processes continue to share the original image.

V. IMPLEMENTATION

Loading an AmberMonkey artifact requires more than native instructions whose runtime accesses pass through the RIT. SpiderMonkey's generators emit instructions into an opaque MacroAssembler buffer while recording artifact metadata against labels within that buffer. This metadata identifies return addresses, debug traps, resume points, profiler sites, and IC transitions. AmberMonkey resolves these labels to code offsets, serializes the offsets with the instructions, and reconstructs the expected JIT objects around the image-backed code. CacheIR records instead preserve the program and stub-layout metadata needed for reconstruction. We obtain these records by reusing SpiderMonkey's code generators through the transparent interception described in Section IV. The following subsections describe the two-build image pipeline, runtime installation and garbage-collection integration, and optimizations that eliminate selected RIT accesses.

A. Image Construction

AmberMonkey constructs the deployed image through a recording build and a final build. The recording build captures artifacts while executing the self-hosted library and tp6-Train. A packing script serializes these artifacts, and the final build embeds the resulting image in the engine library. Figure 3 summarizes this pipeline and subsequent runtime installation.

Each selected artifact contributes native instructions and the metadata needed to reconstruct one SpiderMonkey JIT object. Baseline metadata identifies control-transfer and instrumentation sites by code offset. CacheIR metadata instead

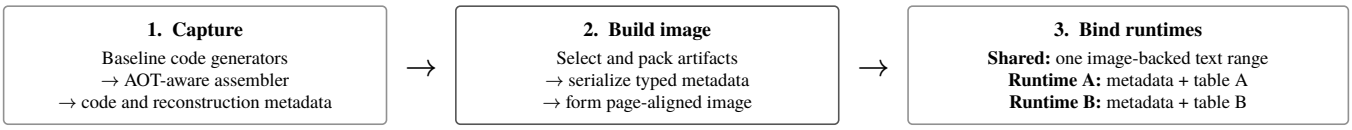

Figure 3: AmberMonkey captures code from SpiderMonkey's existing Baseline code generators, packs selected artifacts into an immutable image, and binds runtime dependencies separately for each runtime. Runtime-private wrappers and tables refer to the same file-backed instruction ranges without copying their bytes.

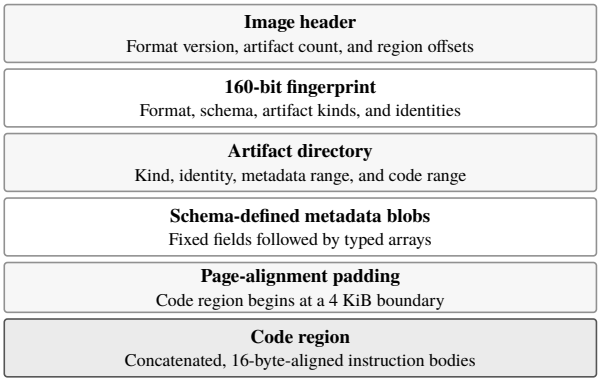

Figure 4: Packed AmberMonkey image. Directory entries map artifact identities to schema-defined metadata and instruction bodies. Page alignment places the immutable code in a distinct region.

preserves the program, stub layout, and tracing information. The packer organizes these records using the offset-based layout in Figure 4. Its directory maps artifact identities to variable-sized metadata blobs and 16-byte-aligned instruction bodies within a page-aligned code region.

AmberMonkey identifies each Baseline function with a 160-bit SHA-1 digest of compiler-visible script state, including flags, frame and IC layout, scope structure, and immutable bytecode metadata. SpiderMonkey's 32-bit script-data hash only filters candidates, while installation requires full-digest equality. An image-wide configuration record separately captures Baseline code-generation settings, Spectre mitigations, and relevant thresholds, which the loader checks before installation. The prototype does not compare canonical inputs after a digest match, so a complete SHA-1 collision remains a limitation.

A declarative YAML schema defines the fixed fields and typed arrays for each artifact kind. The schema generator emits padding-explicit C++ structures, typed encoders and decoders, and compile-time checks for wire sizes, object representations, and artifact-kind numbering. The packer incorporates the explicit format version, schema, artifact kinds, and identities into a 160-bit fingerprint. Schema changes therefore alter the fingerprint automatically, while wire-format changes require an explicit version increment. A schema-defined configuration record also captures code-generation options embedded in the instructions. The loader rejects mismatched configurations, including the three Spectre-mitigation options.

B. Garbage-Collection Integration

AmberMonkey preserves SpiderMonkey's JIT interfaces by representing each image-backed instruction range with a `JitCode` wrapper. The garbage collector manages this wrapper normally, while the static instructions remain owned by

```
.set link_site, 241452
.section .text.aot,"ax",@progbits
.incbin "AOTImage.inc", 0, link_site
.long engine_symbol - . - 4
.incbin "AOTImage.inc", link_site + 4
```

Listing 2: Equivalent assembly for one link site. The two `.incbin` directives omit the recorded 4-byte zero field, which `.long` replaces with a PC-relative relocation.

the image and outlive the wrapper. Ordinary `JitCode` objects carry relocation metadata for tracing embedded pointers and release their dynamically allocated instructions during finalization.

AmberMonkey-backed `JitCode` objects require neither operation. Capture excludes movable garbage-collected pointers, so the collector skips tracing their instructions and finalization discards only the wrapper. The atoms-zone code cache roots image-backed CacheIR wrappers for the runtime's lifetime, while attached IC stubs retain ordinary tracing for their private fields.

C. Optimizations

AmberMonkey replaces some RIT accesses with native-linker relocations when a slot names a non-interposable engine definition. During capture, the assembler leaves a 4-byte PC-relative displacement initialized to zero and records its offset and slot. The packer splits the image around these sites. As shown in Listing 2, the embedding shim copies the surrounding byte ranges with `.incbin` and emits a linker relocation between them. Runtime-generated and host-supplied addresses remain in the RIT.

AmberMonkey also mirrors the interrupt state, JIT stack limit, and count of zones requiring pre-write barriers directly in scalar RIT slots. This removes one dependent load from each check. Mutation paths update the mirrors, using atomic stores when another thread may poll them. A nonzero barrier count falls back to the precise per-zone test. We evaluate linker resolution and value mirroring separately.

D. Implementation Scope

The x86-64 prototype covers the selected corpus and intercepted operand forms. It adds one word to each Baseline or stub frame and a private preamble to each image-backed Baseline artifact. Deterministic trampolines and entry preambles remain generated during initialization. The prototype supports seven permanent atom immediates but rejects movable garbage-collected pointers and retargetable table entries.

The image is a trusted artifact compiled into the engine; an externally replaceable image would require stronger validation and authentication. Profiler or coverage instrumentation can also patch image-backed pages, making the affected pages private and forfeiting their cross-process sharing.

VI. EXPERIMENTAL METHODOLOGY

A. Research Setup

We evaluate AmberMonkey in Firefox 153.0a1 on x86-64. We compile the default and AOT-enabled browsers from separate object directories using Clang and LLD 21.1.8. Both are release builds with `-O2` optimization and debug symbols. Profile-guided optimization (PGO) and link-time optimization (LTO) are disabled in both builds.

Experiments run on an AMD Ryzen 5 7640U with 6 cores, 12 hardware threads, and 32 GB of memory. The machine runs NixOS 26.05 with Linux 7.0.9. Web benchmarks are notoriously noisy [20], so before collecting results we disable frequency boost, select the `performance` frequency governor for every online processor, and take logical processor 3 offline. This processor is the simultaneous multithreading sibling of processor 2. We apply the same controls to every configuration and run no other user workloads during collection.

We use the Raptor harness from the evaluated Firefox tree. Raptor serves Speedometer 3.1 and JetStream 3.0 from the local machine. We disable the Firefox content sandbox in every configuration so that benchmark and AOT environment settings propagate identically to all content processes.

We collect 10 independent browser runs for each configuration and benchmark. Each Speedometer 3.1 run uses one fresh browser process and profile for five page cycles. Each JetStream 3.0 run uses one fresh browser process and profile for one page cycle; JetStream performs its own repetitions within that cycle. We execute configurations in 10 randomized blocks, with each block containing one run of every configuration. Internal benchmark iterations and page cycles are not treated as independent samples.

For each suite, we report the arithmetic mean of its 10 independent scores and a 95% nonparametric bootstrap confidence interval. For pairwise comparisons, we compute a score ratio within each randomized block and report the geometric mean of the 10 ratios with a 95% bootstrap confidence interval. Ratios aggregated across heterogeneous benchmark components also use the geometric mean.

B. Corpus Construction

During the recording build, we run tp6-Train and retain every distinct CacheIR stub body that it attaches. Each record contains the cache kind and CacheIR program that identify the body, plus the field types needed to reconstruct its stub layout. This procedure yields 1758 bodies occupying 732 KB. We exclude application Baseline functions because their identities rarely recur across workloads.

The image also contains all 236 self-hosted Baseline functions, which are available during the build, and the deterministic Baseline Interpreter. The packed image occupies 1 MB. We finalize it before collecting tp6-Test, Speedometer 3.1, and JetStream 3.0 results.

C. Execution Configurations

Table 2 defines the evaluated configurations. Bytecode-only and AOT-only both disable run-time guest-code compilation and retain deterministic bootstrap generation. They differ only in whether the immutable AOT corpus supplies matching artifacts.

Configuration	AOT	Guest-code JIT	Bootstrap generation	AOT miss
Bytecode-only	No	Disabled	Enabled	Interpret
AOT-only	Yes	Disabled	Enabled	Interpret
Default JIT	No	All tiers	Enabled	Compile
Default JIT + AOT	Yes	All tiers	Enabled	Compile

Table 2: Execution configurations. Guest-code JIT includes Baseline-function, CacheIR stub-body, and optimizing compilation. Bootstrap generation comprises deterministic trampolines and entry preambles.

For the indirection-cost ablation, we construct an exact corpus for the measured workload so that every configuration executes the same Baseline artifacts. This corpus is an experimental control rather than a deployable configuration, and we exclude it from held-out coverage results. We compare run-time-generated Baseline code against AOT code with generic indirection, value mirroring, native-linker resolution, and the complete x86-64 lowering.

D. Measurement and Analysis

We report identity coverage as the fraction of unique requested artifact identities present in the AOT image. Request hit rate is the fraction of dynamic artifact requests served by the AOT image. We report both metrics per workload so that one benchmark cannot dominate the suite summary. Generated-code bytes avoided, packed-image size, linked-binary growth, and resident memory are reported as separate quantities.

VII. EVALUATION

We evaluate whether the fixed AOT corpus generalizes to held-out workloads, how much performance it recovers under restricted execution, and what overhead per-runtime indirection introduces. We then report the image's binary-size cost and evaluate its effect on private JIT memory and cross-process sharing.

A. Corpus Coverage

We first measure how well the corpus collected from tp6-Train generalizes to tp6-Test, Speedometer 3.1, and JetStream 3.0. The tp6-Test set contains sites from the same page-load suite that are disjoint from tp6-Train. Speedometer 3.1 models interactive web applications, while JetStream 3.0 contains JavaScript and WebAssembly workloads. None participates in corpus construction.

We report coverage separately for each of the three artifact populations in the image. The Baseline Interpreter is a single deterministic engine artifact that every AOT-using process loads at startup (Table 3). Baseline-function and CacheIR stub coverage are the workload-sensitive quantities and are reported in Table 4 and Table 5. Utilization is the fraction of corpus artifacts referenced during execution; AOT hit rate is the fraction of dynamic requests served by the image. Values are means across three iterations; run-to-run variance is below display precision for these workloads and is omitted.

The tp6-Train IC corpus serves 99.9% of requests on tp6-Test, showing that it generalizes to disjoint sites from the

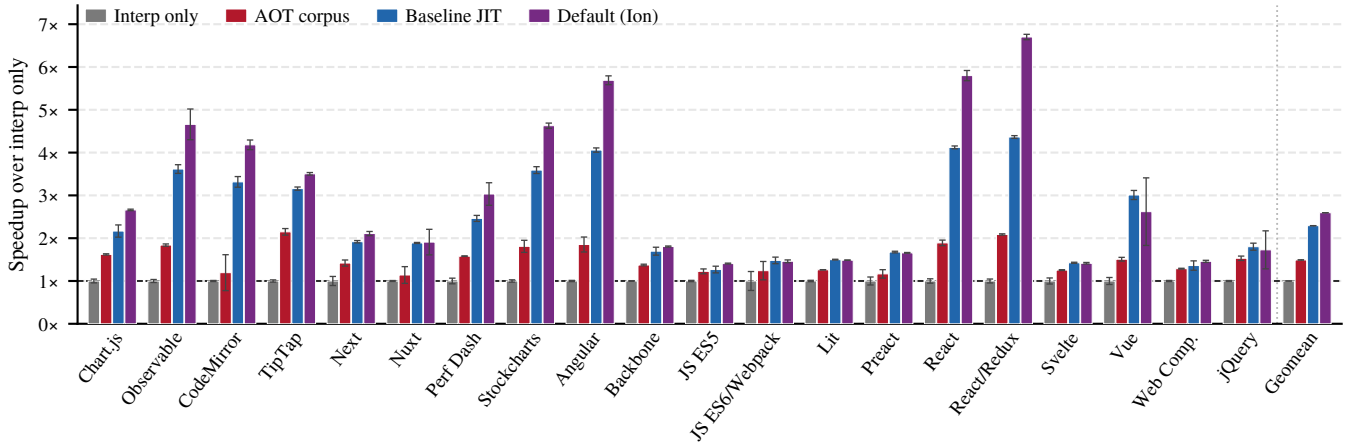

Figure 5: Per-workload Speedometer 3.1 speedup over the interpreter-only baseline for each configuration, with the geometric mean of the 20 workload ratios at the right. AOT corpus is AmberMonkey's restricted-execution image, Baseline JIT is runtime-generated Baseline with Ion disabled, and Default (Ion) is unrestricted SpiderMonkey. Whiskers are the standard deviation of per-run ratios.

Metric	tp6-Test	Speedometer 3.1	JetStream 3.0
Corpus size	1	1	1
Processes loaded	66	7	7

Table 3: Baseline Interpreter coverage on tp6-Test, Speedometer 3.1, and JetStream 3.0. The image ships a single interpreter blob, which every AOT-using content process loads during image attachment.

Metric	tp6-Test	Speedometer 3.1	JetStream 3.0
Corpus size	236	236	236
Installed	71	80	104
Utilization	30.1%	33.9%	44.1%
AOT hit rate	11.7%	43.7%	5.8%

Table 4: Self-hosted Baseline-function coverage on tp6-Test, Speedometer 3.1, and JetStream 3.0. Utilization is the fraction of corpus functions installed at least once; AOT hit rate is the fraction of dynamic Baseline-function requests served by the image.

Metric	tp6-Test	Speedometer 3.1	JetStream 3.0
Corpus size	1,758	1,758	1,758
Attached	1,258	1,090	1,197
Utilization	71.6%	62.0%	68.1%
Total attaches	790.8k	1.25M	1.47M
AOT hit rate	99.9%	99.6%	98.3%

Table 5: CacheIR stub coverage on tp6-Test, Speedometer 3.1, and JetStream 3.0. Utilization is the fraction of corpus stub bodies attached at least once. Total attaches counts every stub-attach request; AOT hit rate is the fraction served by the image, with the remainder resolved by the per-zone stub cache or fresh compilation.

same suite. It serves 99.6% on Speedometer 3.1 and 98.3% on JetStream 3.0. JetStream's lower rate shows slightly less generalization to its scientific and WebAssembly kernels. Baseline-function coverage measures only build-time-known self-hosted functions, not predictions of application functions.

B. Restricted Execution

Figure 5 reports Speedometer 3.1 speedup per workload for each configuration, relative to the interpreter-only baseline; the rightmost group is the geometric mean of the 20 workload ratios and equals the aggregate-score ratio. AOT-only execution reaches 1.51×, runtime-generated Baseline reaches 2.29×, and default SpiderMonkey reaches 2.62×. Both restricted configurations disable guest-code JIT compilation, so their difference isolates the performance recovered by the immutable corpus: AOT-only recovers 66% of Baseline-JIT throughput and reaches 57% of default-tier throughput, a 51% improvement over interpretation.

C. Comparison with V8 Jitless

V8 exposes a `--jitless` flag that disables all run-time code generation, including its Sparkplug baseline compiler, Maglev, and TurboFan optimizing tiers [2]. It serves as the closest production analogue to AmberMonkey's restricted-execution configuration: both prohibit guest-triggered native code emission and rely on precompiled artifacts plus a generic interpreter to execute JavaScript. We compare the two engines on Speedometer 3.1 and JetStream 3.0 to place AmberMonkey's recovered throughput on an absolute cross-engine footing rather than a purely intra-SpiderMonkey ratio.

TODO: Collect V8 Jitless numbers (Chrome/Node with `--jitless`) on the same hardware and Raptor harness. Report per-suite geometric means for (i) V8 default, (ii) V8 Jitless, and (iii) AmberMonkey AOT-only, alongside the Jitless-to-default fraction for each engine so the two restricted configurations can be compared as fractions of their respective unrestricted tiers.

D. AOT Image Installation Cost

AmberMonkey replaces some run-time compilation with image validation, metadata reconstruction, and per-runtime initialization. We measure its end-to-end effect on content-process startup and attribute that cost to the installation phases.

We use Firefox's `cpstartup` benchmark and compare two configurations of the same AOT-enabled binary with the image disabled and enabled; both retain fallback compilation. Each of 10 randomized blocks contains an untimed pair for the primary comparison and a timed pair for attribution and measurement of instrumentation overhead.

Per-content-process timers cover image compatibility, Baseline Interpreter and inline-cache (IC) corpus reconstruction, Runtime Indirection Table (RIT) initialization, lazy Baseline and IC attachment, and residual compilation.

TODO: Report the paired `cpstartup` difference and ratio with 95% bootstrap confidence intervals, per-phase time and call counts, and timer overhead.

E. Indirection Overhead

We measure the steady-state cost of AOT indirection with Linux `perf stat` hardware performance counters. Both configurations disable Ion compilation and execute the same JavaScript source. The runtime configuration generates Baseline machine code during execution, whereas the AOT configuration uses `--aot-only` with an exact corpus containing the measured functions. We count user-mode cycles, retired instructions, and reference cycles under the machine controls described above. The analyzer rejects observations for which the counters are missing or multiplexed.

Each microbenchmark spends nearly all of its execution in a fixed-count hot loop. `perf stat` wraps the complete shell process, so its counts include startup and the initial tier transition rather than sampling only the loop's instruction addresses. The long loop amortizes these fixed costs; dividing by the semantic iteration count yields an estimate that converges on steady-state loop cost. We execute one fresh process per configuration. Figure labels report arithmetic means, and whiskers show one standard deviation when multiple observations are available. User cycles per iteration are the primary cost metric. Retired instructions per iteration identify additional executed work, reference cycles expose frequency drift, and instructions per cycle distinguish extra work from reduced pipeline throughput.

We designed eight microbenchmarks around the generated code that AmberMonkey changes most directly. The five targeted kernels stress the loop interrupt check, function-entry stack check, pre-barrier guard, VM-call path, and local ABI-call path. These sites exercise value mirroring, the one-load VM-call lowering, and native-linker relocation. The three controls exercise integer arithmetic, monomorphic property loads, and dense-array loads, whose hot paths are not directly changed by those optimizations.

Figure 6 reports the per-kernel measurements and their geometric mean. The preliminary pass requires 1.07× as many cycles for AOT Baseline code as for runtime-generated Baseline code, an overhead of 7%. With only one process per configuration, we use this result to validate the measurement and presentation pipeline rather than as the final estimate.

Retired instructions and instructions per cycle (IPC) resolve where the cycle cost comes from. AOT Baseline code retires 16% more instructions per iteration than runtime-generated Baseline code, reflecting the extra loads that resolve runtime pointers through the indirection table and mirrored scalars. Under the same load, AOT execution reaches 9% higher IPC, indicating that the added instructions occupy previously idle pipeline slots rather than extending the critical path. The cycle overhead is therefore an instruction-count effect that partial front-end and load-store parallelism absorb, not a loss of throughput on the original work.

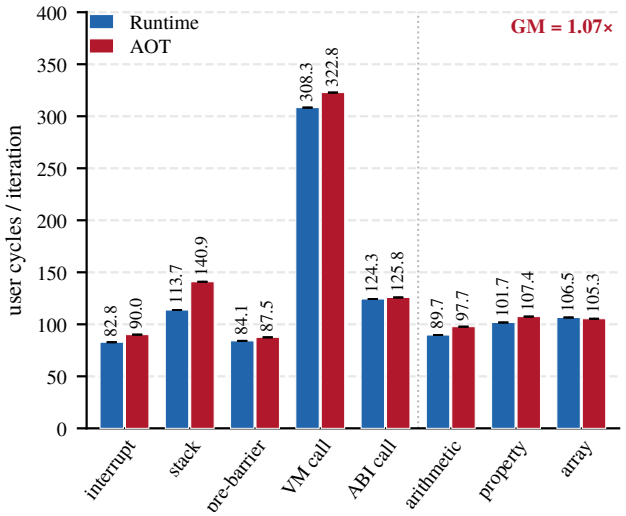

Figure 6: Preliminary User-mode cycles per iteration for runtime-generated and AOT Baseline code with Ion disabled (one fresh process per configuration). Labels give arithmetic means; whiskers show one standard deviation when multiple observations are available. The dotted rule separates five optimization-sensitive sites from three controls. GM is the geometric mean of the eight AOT/runtime ratios.

F. Cross-Process Memory Sharing

We instrument Speedometer 3.1 content processes with a `smaps` sidecar that separates each process's executable memory into `.text.aot` (file-backed libxul pages) and anon-exec (private JIT pages). Table 6 reports the Peak sample across three iterations.

Under AOT-only, `.text.aot` has a 5.3× RSS/PSS ratio across 9 content processes. Adding the image increases its total RSS by 23.5 MB but its PSS by only 2.3 MB, confirming that processes share its physical pages. Against the tier-matched, Ion-disabled runtime-Baseline configuration, AOT-only reduces engine PSS per process from 14.3 MB to 6.19 MB, a 57% reduction. We do not compare this reduction against the default configuration, whose Ion tier changes the code profile.

G. Binary Size

AmberMonkey increases the deployed Firefox engine binary by 1.04 MB (0.56%), from 186.28 MB to 187.32 MB. We measure the file-backed `PT_LOAD` segments of `libxul.so` with `readelf` in matched release builds; the Firefox launcher does not contain SpiderMonkey. The embedded AOT image occupies 1 MB, measured between `aot_image_start` and `aot_image_end`, while the remaining 36 KB comprises other linked code and alignment overhead. The linker folds the `.text.aot` input section into the executable `.text` output section.

VIII. RELATED WORK

Partial evaluation can derive compilation by specializing an interpreter to a fixed program [4]. SpiderMonkey's Portable

config	content procs	.text.aot RSS (MB)	.text.aot PSS (MB)	RSS/PSS (sharing)	anon-exec PSS (MB)	engine PSS / proc (MB)
interp-only	18	499.91	49.81	10.04	0	2.77
default	9	270.41	53.56	5.05	72.55	14.01
default-no-ion	9	271.96	53.55	5.08	75.16	14.3
aot	9	277.66	54.44	5.1	69.06	13.72
aot-corpus	9	293.88	55.83	5.26	1.89	6.19

Table 6: Speedometer 3.1 engine memory at Peak, per configuration. Values are means across three iterations. `.text.aot RSS/PSS` is the cross-process sharing ratio; higher is more shared. `anon-exec PSS` is total private-JIT memory across content processes. `engine PSS / proc` is (`.text.aot PSS` + `anon-exec PSS`) / n_procs.

Baseline Interpreter (PBL) executes JavaScript bytecode with CacheIR; Weval specializes PBL for bytecode already present in a WebAssembly snapshot and combines it with AOT IC stubs [5]. AmberMonkey instead builds its image before guest programs are known, so it selects engine artifacts that are deterministic or recur across workloads rather than specializing guest bytecode.

Reusable Inline Caching (RIC) carries context-independent IC information from one execution into later executions and eagerly repopulates IC state to avoid cold misses [21]. AmberMonkey currently attaches stubs only after a runtime observation. Its AOT corpus makes eager attachment easier because the native body is already materialized; the remaining problem is reconstructing site-specific fields such as shapes and slot offsets.

ShareJIT provides Android Runtime processes with a global cache of runtime-compiled methods and limits context-dependent optimization to improve sharing [22]. AmberMonkey instead freezes selected Baseline artifacts during a trusted build. Firefox processes map them through the engine library without coordinating cache ownership or adding guest-generated instructions, while private metadata and indirection tables supply per-runtime state.

GraalVM Native Image compiles reachable Java code under a closed-world assumption, while Dart AOT produces architecture-specific machine code for a specified Dart program [23], [24]. Both know the application at build time. AmberMonkey instead targets a browser engine whose guest programs arrive after deployment and therefore precompiles reusable engine artifacts rather than complete applications.

V8′s embedded builtins place isolate-independent generated code in the engine binary and keep isolate-specific state separate, eliminating private builtin copies [11]. V8 Jitless instead disables run-time executable memory and executes JavaScript without its JIT tiers [2]. AmberMonkey applies file-backed sharing to Baseline artifacts and uses a fixed corpus to recover native execution under a similar restriction.

IX. CONCLUSION

We have presented AmberMonkey, a generic formulation of AOT compilation for a Baseline tier. Our preliminary evaluation indicated that three categories of artifact may be ammenible to AOT compilation for separate reasons. IC bodies recur across workloads. This recurrence enables a small corpus to achieve high dynamic coverage on unseen workloads. In contrast, the distribution for Baseline compiled functions we found much sparser across workloads. Nonehteless, we identified a ubiquitous corpus of self-hosted code, including JavaScript builtins, which we included in our corpus. Lastly we identified deterministic artifacts, namely the Baseline Interpreter, which we provided AOT to avoid redundant recompilations across processes.

X. FUTURE WORK

A model which decouples the JIT copmiling process from a respective consumer process offers an interesting angle for offline optimization.

1. *Eager IC stub attachment*: By using the initial engine invocation to collect profiling information regarding which IC stubs attach at particular script locations, we may be able to skip expsenive fallback stub routines through eager attachment. The utility of avoiding the fallback stub was elucidated by Choi et al [21]. An AOT format for IC stubs makes materializing the IC stub bodies convenient, however, a serializable format for stub paramaters such as Object Shapes forms a technical barrier.

2. *Type Specialized Builtins*: Previous work done with V8′s Torque DSL has demonstrated that type-specialized fast paths can improve the peformance of JavaScript builtins. Common operand-type patterns can establish early control flows independent from generic handlers, allowing more aggressive specialziation for Baseline code.

Bibliography

[1] S. Groß, "State of JavaScript Engine Exploitation." [Online]. Available: https://saelo.github.io/presentations/calif_community_talk_2026_state_of_js_engine_exploitation.pdf

[2] J. Gruber, "JIT-less V8." [Online]. Available: https://v8.dev/blog/jitless

[3] M. Serrano, "Of JavaScript AOT Compilation Performance," *Proceedings of the ACM on Programming Languages*, vol. 5, no. ICFP, 2021.

[4] Y. Futamura, "Partial Evaluation of Computation Process—An Approach to a Compiler-Compiler," *Systems, Computers, Controls*, pp. 45–50, 1971.

[5] C. Fallin and M. Bernstein, "Partial Evaluation, Whole-Program Compilation," *Proceedings of the ACM on Programming Languages*, vol. 9, no. PLDI, 2025.

[6] J. Kupoluyi *et al.*, "Muzeel: Assessing the Impact of JavaScript Dead Code Elimination on Mobile Web Performance," in *Proceedings of the 22nd ACM Internet Measurement Conference*, ACM, 2022, pp. 335–348.

[7] Mozilla, "SpiderMonkey." [Online]. Available: https://firefox-source-docs.mozilla.org/js/

[8] T. Verwaest and M. Hillerström, "Blazingly Fast Parsing, Part 2: Lazy Parsing." [Online]. Available: https://v8.dev/blog/preparser

[9] J. de Mooij, M. Gaudet, C. Ireland, N. Henderson, and J. N. Amaral, "CacheIR: The Benefits of a Structured Representation for Inline Caches," in *Proceedings of the 20th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (MPLR)*, ACM, 2023.

[10] Mozilla, "Raptor Test List (tp6 pageload suite)." [Online]. Available: https://firefox-source-docs.mozilla.org/testing/perfdocs/test-list.html

[11] J. Gruber, "Embedded builtins." [Online]. Available: https://v8.dev/blog/embedded-builtins

[12] P. Kocher *et al.*, "Spectre Attacks: Exploiting Speculative Execution," in *2019 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2019.

[13] B. Holley, "The Zero-Days Are Numbered." [Online]. Available: https://blog.mozilla.org/en/firefox/privacy-security/ai-security-zero-day-vulnerabilities/

[14] Nebula Security, "IonStack Part I: Unsound IonBanana Peel in Ion Compiler, Slipping Through Firefox's SpiderMonkey JIT." [Online]. Available: https://nebusec.ai/research/ionstack-part-1-cve-2026-10702/

[15] N. Wang and Z. Chen, "Enhanced Insecurity Mode: 23 RCEs in Edge's 'Safe' WebAssembly Interpreter." [Online]. Available: https://www.offensivecon.org/speakers/2026/nan-wang-and-ziling-chen.html

[16] Mozilla, "How SpiderMonkey Optimizes." [Online]. Available: https://firefox-source-docs.mozilla.org/js/how-we-optimize.html

[17] Mozilla, "Inline Caches / CacheIR." [Online]. Available: https://firefox-source-docs.mozilla.org/js/cacheir.html

[18] Mozilla, "Process Model." [Online]. Available: https://firefox-source-docs.mozilla.org/dom/ipc/process_model.html

[19] Tool Interface Standards Committee, "Tool Interface Standard (TIS) Executable and Linking Format (ELF) Specification, Version 1.2," 1995. [Online]. Available: https://refspecs.linuxbase.org/elf/elf.pdf

[20] Z. Toufie and B. Kabaso, "OS Noise Mitigations for Benchmarking Web Browser Execution Environment Performance," *Discover Computing*, vol. 27, p. 37, 2024.

[21] J. Choi, T. Shull, and J. Torrellas, "Reusable Inline Caching for JavaScript Performance," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2019.

[22] X. Xu, K. D. Cooper, J. Brock, Y. Zhang, and H. Ye, "ShareJIT: JIT Code Cache Sharing across Processes and Its Practical Implementation," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, 2018.

[23] GraalVM, "GraalVM Native Image." [Online]. Available: https://www.graalvm.org/latest/reference-manual/native-image/

[24] Dart, "dart compile." [Online]. Available: https://dart.dev/tools/dart-compile

XI. DRAFT NOTES

NOTE: 1) The 7% indirection overhead: This is done on some micro-benchmarks which specfically exercise the areas affected by AmberMonkey. Should we do a stock: `--no-ion` v.s AOT: `--aot-only` over Speedometer3 to isolate E2E Baseline tier indirection overhead? We will likely get a much lower number than 7%. Maybe 2-3%. Chase and I both found around 5% prior to slight 'optimziations'.

NOTE: 2) Getting obvious stuff out of the way: Must run any performance benchmark with n=20 once settled upon. Experimental methodology lays out some principles I don't follow yet.

NOTE: 4) We are missing performance metrics for how long AOT loading takes: deserialization, time filling the RIT (Runtime Indirection Table), etc.

NOTE: 5) Related work is super incomplete.

NOTE: 6) The Restricted Execution Performance (the main benchmark) for AmberMonkey is less strong than Ihoped for. We also have not yet turned off WASM and Regex. this could be bad.

NOTE: 7) Figure 2. is low effort for now. I'd like to design the papers CENTRAL FIGURE here.

NOTE: 8) The execution configurations introduced in the methodology don't necessarily line up with those used in the evaluation. There is generally too much inconsitency around what execution modes are used and their abbreviations.

NOTE: 9) Corpus coverage for only JetStream and Speed3 is insufficient. Should add 1 or two more. Octane is a subset of JetStream unforunately. Sunspider old.

NOTE: 11) The paper revolves around finding two baseline artifacts: IC stubs and Baseline compiled builtins, which work AOT for different reasons. Yet we only ever evaluated them in conjunction. I have separated their contributions months ago and its like 97% ICs and 3% self-hosted perf contribution.

NOTE: 12) Can we call using the linker an optimization? Hmmm... 'Value Mirroring' sounds cool, but it breaks any GOT hardening esque hardening we could perform.

NOTE: 15) There is a AI generated CGO review in an adjacent file.